

Spring Security — do Zero ao Deploy

Apostila completa para implementar autenticação e autorização com Spring Security 6, usando o projeto **_SistemaLogin** como base real de aprendizado. Passo a passo, do pom.xml ao HTML final.

Spring Boot 3.4.5

Java 17

Spring Security 6

Thymeleaf

MySQL + BCrypt

1 O que é Spring Security?

Spring Security é o framework de segurança padrão do ecossistema Spring. Ele cuida de autenticação (verificar *quem* é o usuário), autorização (verificar *o que* o usuário pode fazer), proteção CSRF e gerenciamento de sessão.

É adotado em aplicações corporativas e acadêmicas por ser robusto, altamente configurável e integrado nativamente ao Spring Boot.

Analogia

Pense no Spring Security como o porteiro de um prédio: ele verifica o crachá (autenticação) e decide quais andares você pode acessar (autorização). Sem configuração, ele bloqueia *tudo*.

O que ele protege automaticamente?

PILAR 1

Autenticação

Verifica a identidade do usuário via formulário, JWT, OAuth2 ou Basic Auth.

PILAR 2

Autorização

Controla o acesso por rota e por role. Ex: só ADMIN acessa `/admin/**`.

PILAR 3

CSRF

Proteção contra Cross-Site Request Forgery em formulários POST.

PILAR 4

Sessão

Gerencia sessões ativas, timeout e controle de sessões simultâneas.

PILAR 5

Headers HTTP

Adiciona headers de segurança: X-Frame-Options, HSTS, CSP e outros.

Atenção ao adicionar a dependência

Assim que você adicionar `spring-boot-starter-security`, o Spring bloqueia **todas as rotas**

automaticamente. Você precisa configurar o `SecurityConfig` para liberar o que for necessário.

2 Configurando o pom.xml

Este é o `pom.xml` do projeto **_SistemaLogin**. Cada dependência tem um papel específico na cadeia de segurança.

```
<!-- Spring Boot 3 exige Java 17 como mínimo -->
<parent>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.4.5</version>
</parent>

<dependencies>

  <!-- 1. JPA: faz a ponte entre Java e o banco de dados -->
  <dependency>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- 2. SECURITY: o coração desta apostila -->
  <dependency>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- 3. THYMELEAF: motor de templates HTML -->
  <dependency>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>

  <!-- 4. THYMELEAF + SECURITY: habilita tags sec: no HTML -->
  <dependency>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
  </dependency>

  <!-- 5. MySQL: driver do banco de dados -->
  <dependency>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- 6. LOMBOK: elimina getters/setters/construtores -->
  <dependency>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

</dependencies>
```

Dependência	Para que serve
<code>starter-security</code>	Núcleo — autentica e autoriza tudo
<code>starter-data-jpa</code>	Persiste e consulta usuários no banco
<code>starter-thymeleaf</code>	Renderiza as páginas HTML no servidor
<code>thymeleaf-extras-springsecurity6</code>	Habilita tags <code>sec:</code> nos templates
<code>mysql-connector-j</code>	Driver JDBC para conexão com MySQL
<code>lombok</code>	Gera getters, setters e construtores

3 Arquitetura e Filter Chain

O Spring Security funciona como uma **cadeia de filtros** (Security Filter Chain) que intercepta cada requisição HTTP *antes* de ela chegar ao seu Controller. Cada filtro verifica um aspecto diferente da segurança.

Fluxo de uma requisição

1

Requisição HTTP chega

O navegador faz uma requisição GET ou POST para qualquer rota da aplicação.

2

Security Filter Chain intercepta

Antes de chegar ao Controller, a requisição passa por todos os filtros de segurança configurados.

3

Verificação de autenticação

O filtro verifica se há uma sessão ativa ou credenciais válidas no request.

4

Verificação de autorização

Com base nas regras do `SecurityFilterChain`, verifica se o usuário tem permissão para acessar aquela rota específica.

5

Controller é chamado

Somente se todas as verificações passarem, a requisição chega ao seu Controller e retorna a View.

Os 4 blocos que você vai criar

Bloco	Classe	Responsabilidade
Entidade	<code>Usuario</code>	Representa o usuário no banco de dados
Repositório	<code>UsuarioRepository</code>	Busca o usuário no banco via email
Serviço	<code>UsuarioDetailsService</code>	Ponte entre o banco e o Spring Security
Configuração	<code>SecurityConfig</code>	Define todas as regras de segurança

Spring Boot 3 vs Spring Boot 2

No Spring Boot 2, a configuração era feita herdando `WebSecurityConfigurerAdapter`. No Spring Boot 3 essa classe foi

removida

. A forma correta agora é usar beans com `@Bean` e lambdas — como mostrado nesta apostila.

4 Passo 1 — Entidade Usuario

Este é o modelo mapeado para a tabela no banco. O Spring Security precisa de pelo menos: **username**, **password** e **roles** para funcionar.

```

@Data // Lombok: getters, setters, equals, hashCode
@NoArgsConstructor // Lombok: construtor sem args (obrigatório para JPA)
@Entity
@Table(name = "usuarios")
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String senha; // sempre armazenar o hash BCrypt

    @Column(nullable = false)
    private String role; // ex: "ROLE_USER" ou "ROLE_ADMIN"
}

```

ROLE_USER

Usuário comum

- Acessa páginas públicas
- Acessa `/home` após login
- Não acessa `/admin/**`
- Role atribuída no cadastro

ROLE_ADMIN

Administrador

- Acessa todas as páginas
- Acessa `/admin/**`
- Pode gerenciar usuários
- Role atribuída manualmente

💡 Por que o prefixo ROLE_?

O Spring Security espera roles começando com `ROLE_`. Ao usar `.hasRole("USER")` no `SecurityConfig`, o Spring adiciona `ROLE_` automaticamente. Com `.hasAuthority("ROLE_USER")` você precisa escrever o nome completo.

5 Passo 2 — Repository

O Repository é a interface que faz as consultas no banco. Você só precisa de um método customizado: buscar o usuário pelo email. O Spring Data JPA cria a query SQL automaticamente pelo nome do método.

```
@Repository
public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

    // Spring Data cria automaticamente a query:
    // SELECT * FROM usuarios WHERE email = ?
    Optional<Usuario> findByEmail(String email);
}
```

✓ Por que Optional?

Evita `NullPointerException`. Se o usuário não existir, retorna `Optional.empty()` em vez de `null`, permitindo encadear `.orElseThrow()` com uma mensagem de erro clara.

6 Passo 3 — UserDetailsService

Esta é a **peça central** da integração. Você está dizendo ao framework: "*quando precisar autenticar alguém, use este método para buscar o usuário.*" O Spring Security não sabe como seu banco está estruturado — você ensina aqui.

```

@Service
public class UsuarioDetailsService
    implements UserDetailsService {

    @Autowired
    private UsuarioRepository usuarioRepository;

    // 0 Security chama este método com o "username" digitado no form
    @Override
    public UserDetails loadUserByUsername(String email)
        throws UsernameNotFoundException {

        // 1. Busca no banco pelo email
        Usuario usuario = usuarioRepository
            .findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuário não encontrado: " + email
            ));

        // 2. Converte a role para o formato do Spring Security
        SimpleGrantedAuthority authority =
            new SimpleGrantedAuthority(usuario.getRole());

        // 3. Retorna UserDetails – objeto padrão do framework
        return new User(
            usuario.getEmail(),    // username
            usuario.getSenha(),    // senha já em hash BCrypt
            Collections.singletonList(authority)
        );
    }
}

```

🔧 O papel do UserDetailsService

É a "ponte" entre o seu banco e o Spring Security. O framework não sabe como seus dados estão estruturados — ele chama `loadUserByUsername()` e espera receber um `UserDetails`. Você decide como buscar e montar esse objeto.

Esta é a classe mais importante. Ela define os 3 beans essenciais que governam toda a segurança da aplicação.

Bean 1 — PasswordEncoder (BCrypt)

```
@Bean
public PasswordEncoder passwordEncoder() {
    // BCrypt gera um hash diferente a cada chamada,
    // mesmo para a mesma senha
    return new BCryptPasswordEncoder();
}
```

Bean 2 — DaoAuthenticationProvider

```
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider provider =
        new DaoAuthenticationProvider();
    provider.setUserDetailsService(usuarioDetailsService);
    provider.setPasswordEncoder(passwordEncoder());
    return provider;
}
```

Bean 3 — SecurityFilterChain (regras de acesso)

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
    throws Exception {

    http
        // 1. REGRAS POR ROTA
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/cadastro", "/login", "/css/**")
                .permitAll()           // rotas públicas
            .requestMatchers("/admin/**")
                .hasRole("ADMIN")     // só ROLE_ADMIN
            .anyRequest().authenticated() // resto exige login
        )

        // 2. FORMULÁRIO DE LOGIN
        .formLogin(form -> form
            .loginPage("/login")
            .loginProcessingUrl("/login")
            .defaultSuccessUrl("/home", true)
            .failureUrl("/login?error=true")
            .permitAll()
        )

        // 3. LOGOUT
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout=true")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
            .permitAll()
        );

    return http.build();
}
```

⚠ Ordem das regras importa!

As regras do `authorizeHttpRequests` são avaliadas na ordem em que foram declaradas. Sempre coloque as regras mais específicas antes das mais genéricas.

`.anyRequest().authenticated()` deve ser sempre o último.

O Controller trata as requisições HTTP. Para o sistema de login, você precisa das rotas de cadastro, de exibição do login (o **POST é tratado pelo Spring Security**, não pelo Controller) e da página protegida.

```

@Controller
public class UsuarioController {

    @Autowired private UsuarioRepository usuarioRepository;
    @Autowired private PasswordEncoder passwordEncoder;

    // GET /login → exibe a página de login
    @GetMapping("/login")
    public String login() { return "login"; }

    // GET /cadastro → exibe o formulário
    @GetMapping("/cadastro")
    public String cadastro() { return "cadastro"; }

    // POST /cadastro → processa e salva o usuário
    @PostMapping("/cadastro")
    public String salvarUsuario(
        @RequestParam String email,
        @RequestParam String senha, Model model) {

        if (usuarioRepository.findByEmail(email).isPresent()) {
            model.addAttribute("erro", "E-mail já cadastrado!");
            return "cadastro";
        }

        Usuario novo = new Usuario();
        novo.setEmail(email);
        novo.setSenha(passwordEncoder.encode(senha)); // ← SEMPRE encodar!
        novo.setRole("ROLE_USER");
        usuarioRepository.save(novo);
        return "redirect:/login?cadastro=true";
    }

    // GET /home → página protegida (só logados)
    @GetMapping("/home")
    public String home() { return "home"; }
}

```



Regra de ouro — nunca salve senha em texto puro

Use sempre `passwordEncoder.encode(senha)` antes de persistir no banco. Salvar senha em texto puro é uma falha grave e irreversível de segurança. O banco deve armazenar apenas o hash BCrypt.

Os templates HTML usam dois namespaces: `th:` do Thymeleaf para lógica de template e `sec:` para controle de exibição baseado em autenticação.

login.html — campos obrigatórios

⚠ Nomes de campos obrigatórios

Os campos do form de login

devem

se chamar `name="username"` e `name="password"` — esse é o padrão esperado pelo Spring Security. Se usar nomes diferentes, o login nunca vai funcionar.

```
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">

<!-- Mensagens de feedback -->
<div th:if="${param.error}">E-mail ou senha inválidos!</div>
<div th:if="${param.logout}">Você saiu com sucesso.</div>

<!-- Spring Security intercepta este POST – sem método no Controller -->
<form th:action="@{/login}" method="post">
  <input type="email"    name="username" required/>
  <input type="password" name="password" required/>
  <button type="submit">Entrar</button>
</form>
```

```
<html xmlns:sec="http://www.thymeleaf.org/extras/spring-security">

<!-- Exibe o email do usuário logado -->
Bem-vindo, <span sec:authentication="name"></span>!

<!-- Visível apenas para ROLE_ADMIN -->
<div sec:authorize="hasRole('ADMIN')">
  <p>Painel de Administração</p>
</div>

<!-- Logout DEVE ser POST – proteção CSRF -->
<form th:action="@{/logout}" method="post">
  <button type="submit">Sair</button>
</form>
```



Por que o logout é POST?

O Spring Security exige POST para o logout como proteção CSRF. Um link GET poderia ser explorado para deslogar alguém sem consentimento. Nunca use `<a href="/logout">` .

10

Passo 7 — application.properties

Configuração da conexão com o MySQL, do Hibernate e do Thymeleaf para o ambiente de desenvolvimento.

```
# — Conexão MySQL —
spring.datasource.url=jdbc:mysql://localhost:3306/sistema_login?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=sua_senha
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# — JPA / Hibernate —
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# — Thymeleaf —
spring.thymeleaf.cache=false
```

Valor de ddl-auto	Comportamento	Quando usar
update	Cria/atualiza tabelas sem apagar dados	Desenvolvimento
create	Recria as tabelas do zero a cada restart	Testes locais
validate	Valida o schema sem alterar nada	Produção (seguro)
none	Sem nenhuma ação do Hibernate	Produção

11 Fluxo Completo de Autenticação

O que acontece internamente entre o clique em "Entrar" e o redirecionamento para `/home`.

1

Usuário submete o formulário

POST `/login` com campos `username` e `password` — interceptado pelo Spring Security antes de qualquer Controller.

2

UsernamePasswordAuthenticationFilter

Extraí as credenciais do request e cria um token de autenticação para processar.

3

DaoAuthenticationProvider

Chama `loadUserByUsername(email)` no `UsuarioDetailsService`, que busca o usuário no banco via `UsuarioRepository`.

4

Verificação BCrypt

O `BCryptPasswordEncoder` compara a senha digitada com o hash armazenado no banco. Nunca compara texto puro.

5

SecurityContext atualizado

O usuário fica "logado" na sessão HTTP.

`SecurityContextHolder.setAuthentication(...)` é chamado internamente.

6

Redirect para /home

Configurado em `defaultSuccessUrl("/home", true)` no `SecurityConfig`. Em caso de falha, vai para `/login?error=true`.

12

Erros Comuns e Como Resolver

Os 5 problemas mais frequentes ao configurar Spring Security pela primeira vez.

✗ ERRO 1

There is no PasswordEncoder mapped for the id "null"

Causa: A senha foi salva sem encodar, ou o bean `PasswordEncoder` não foi configurado no `SecurityConfig`.

Solução: Garanta que existe o bean `BCryptPasswordEncoder` e que usa `passwordEncoder.encode(senha)` ao salvar o usuário no Controller.

✗ ERRO 2

403 Forbidden em CSS, JS e imagens

Causa: O Spring Security está bloqueando os arquivos estáticos da aplicação.

Solução: Adicione no

```
SecurityFilterChain : .requestMatchers("/css/**", "/js/**", "/images/
**").permitAll()
```

✗ ERRO 3

Login sempre retorna erro mesmo com credenciais corretas

Causas mais comuns: (1) campo do form não se chama `username` / `password`; (2) senha no banco não está em BCrypt; (3) `loadUserByUsername` busca pelo campo errado.

Como verificar: A senha BCrypt sempre começa com `$2a$`. Se no banco estiver texto puro, o problema está no cadastro.

✗ ERRO 4

Logout via link GET não funciona (403)

Causa: O Spring Security exige POST para logout por proteção CSRF.

Solução: Use sempre `<form method="post" th:action="@{/logout}">`. Nunca use `<a href="/logout">`.

✗ ERRO 5

WebSecurityConfigurerAdapter — cannot find symbol

Causa: Você está seguindo um tutorial do Spring Boot 2.x. Essa classe foi removida no Spring Boot 3.

Solução: Use o estilo moderno com `SecurityFilterChain` como `@Bean`, conforme mostrado nesta apostila.

✓ Resumo das classes criadas

- **model/Usuario.java**
 - Entidade JPA com email, senha (BCrypt) e role
- **repository/UsuarioRepository.java**

- Interface com `findByEmail()`
- **service/UsuarioDetailsService.java**
 - Implementa `UserService`
- **config/SecurityConfig.java**
 - 3 beans: PasswordEncoder, AuthProvider, FilterChain
- **controller/UsuarioController.java**
 - GET/POST /cadastro, GET /login, GET /home
- **templates/login.html**
 - Form com campos `username` e `password`
- **templates/home.html**
 - Tags `sec:authentication` e `sec:authorize`